

# Predictive Parsing (2)

Sam Ogden

# DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

# Learning outcomes

- After this lecture, you should be able to:
- derive parse trees from a BNF grammar
- define what it means for a grammar to be “ambiguous”
- use two additional rules for transforming a grammar

# Parse trees

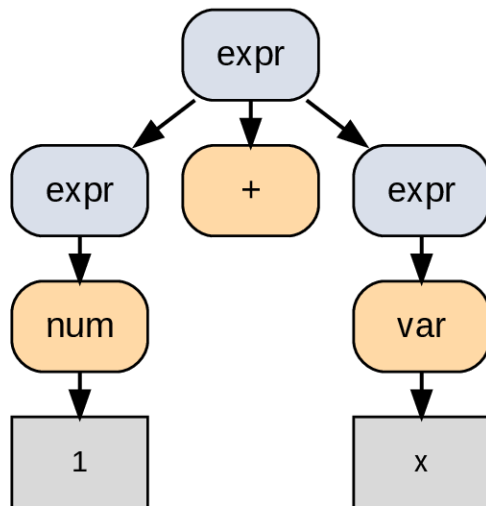
$\text{expr} ::= \text{num} \mid \text{var} \mid \text{expr} + \text{expr}$

Parse 1 + x

# Parse trees

**expr ::= num | var | expr+expr**

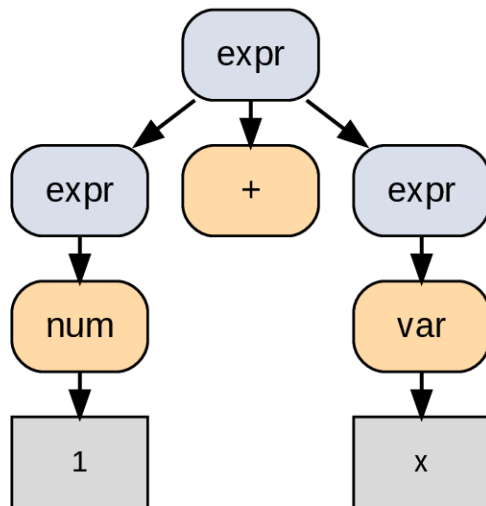
Parse 1 + x



# Parse trees

**expr ::= num | var | expr+expr**

Parse 1 + x



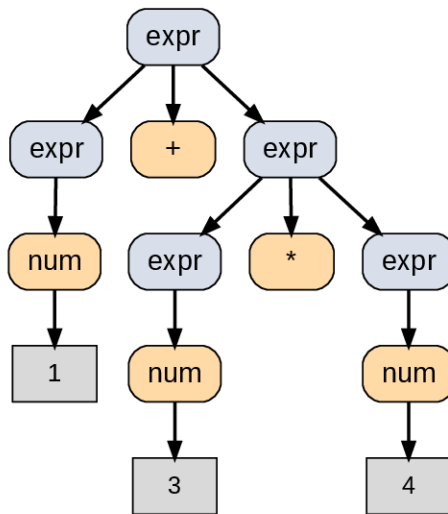
Rules:

1. root is labeled with the start symbol
2. the symbols in one of its productions become child nodes
3. continue until all leaf nodes are terminal symbols

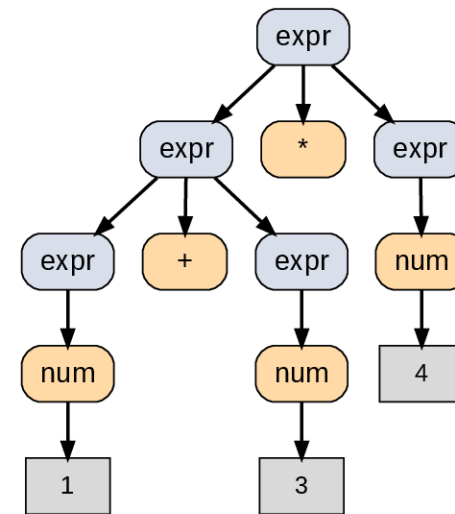
# Ambiguous grammar

$\text{expr} ::= \text{num} \mid \text{expr} + \text{expr} \mid \text{expr} * \text{expr}$

Create a parse tree for  $1 + 3 * 4$ :



$1 + (3 * 4)$



$(1 + 3) * 4$

# Ambiguous grammar

- A grammar is **ambiguous** if, for some string in its language, there is more than one parse tree.
- Different parse trees usually suggest different meanings.
- So: we usually want unambiguous grammars.



# Review: predictive parsing

```
stmt ::= print(expr) | id=expr  
expr ::= id | num
```

- We need to be able to pick a production of a non-terminal based on the next token.

```
1 void stmt() {  
2     switch (token) {  
3         case PRINT:  
4             match(PRINT); match("("); expr(); match(")");  
5             break;  
6         case ID:  
7             match(ID); match("="); expr();  
8             break;  
9         default:  
10            error("syntax error");  
11    }  
12 }
```

# Transforming a BNF grammar

- In the last lecture we learned how to deal with:
- left recursion
- empty productions

Two more ways to transform BNF:

- left factoring
- expanding a non-terminal

*It's important to get comfortable with modifying grammars.*

# Left factoring

$A ::= aB \mid aC \mid bD$

# Left factoring

$A ::= aB \mid aC \mid bD$

How about this replacement? Is it the same?

$A ::= aE \mid bD$   
 $E ::= B \mid C$

# Exercise

Suppose an expression can be a variable or a function call.

- Examples: `tot` (variable), `max(y)` (function call)

BNF:

```
expr ::= var(expr) | var
```

Exercise: Apply left factoring.

# Exercise

Suppose an expression can be a variable or a function call.

- Examples: `tot` (variable), `max(y)` (function call)

BNF:

```
expr ::= var(expr) | var
```

Exercise: Apply left factoring.

# Solution

```
expr ::= var expr1  
expr1 ::= (expr) | ""
```

# Expanding a non-terminal

```
A ::= aB | E  
E ::= cC | dD
```

Any problems with a predictive parser here?

# Expanding a non-terminal

```
A ::= aB | E  
E ::= cC | dD
```

Any problems with a predictive parser here?

## Solution

```
A ::= aB | cC | dD
```



# Rewriting BNFs

- For predictive parsers, we want that, for every non-terminal in the grammar:
- there's only one production for it, or
- each production starts with a terminal token (maybe `"` on its own) and each of those tokens are different

# Exercise

```
prog ::= stmt | stmt;prog  
stmt ::= ID=expr | ID(expr)
```

- There are two problems. What are they?
- Apply left factoring.

# Solution

```
prog ::= stmtprog1  
prog1 ::= ;prog | ""  
stmt ::= IDstmt1  
stmt1 ::= =expr | (expr)
```

# Exercise

Expand a non-terminal (inline `stmt` into `prog`):

```
prog ::= stmtprog1
prog1 ::= ;prog | ""
stmt ::= IDstmt1
stmt1 ::= =expr | (expr)
```

# Solution

```
prog ::= IDstmt1prog1  
prog1 ::= ;prog | ""  
stmt1 ::= =expr | (expr)
```

Note: there is no longer a `stmt` non-terminal.

# Fixed grammar

When expanding non-terminals, verify you didn't change the language.

Example check:

- Original `prog ::= stmt | stmt ; prog` does **not** derive `ID = expr ;` (no empty statement at the end).
- So the transformed grammar should not allow a trailing `;` either.

```
prog ::= IDstmt1prog1
prog1 ::= ;prog | ""
stmt1 ::= =expr | (expr)
```

Derive ID = expr ;

grammar 1:

prog ::= stmt | stmt ; prog

stmt ::= ID = expr | ID ( expr )

prog -> stmt -> ID = expr -> FALSE

prog -> stmt ; prog -> ID = expr ; prog -> FALSE

Can't produce a ; at the end. No empty statement.

|

If we change stmt ; prog to stmt ; prog1

Then we make prog1 ::= "" | stmt

I believe they will be the same then. Deriving the solution below.

NEW SOLUTION WITH THE ADDITION OF RULES

prog -> stmt ; prog1 -> ID = expr ; prog1 -> ID = expr ; = TRUE

grammar 2:

prog ::= ID stmt1 ; prog1

prog1 ::= "" | ID stmt

stmt ::= = = expr | ( expr )

prog -> ID stmt1 ; prog1 -> ID = expr ; prog1 -> ID = expr ; -> TRUE

If we get rid of "" in prog1 for grammar 2 then it would be the same.

I have derived the new test below

NEW SOLUTION WITH REDUCTION OF RULES

prog -> ID stmt1 ; prog1 -> ID = expr ; prog1 -> FALSE

Fixed Grammar by an old TA

# Summary

- A parse tree shows how a string can be derived from a BNF grammar.
- To use predictive parsing, our BNF syntax must be in a certain form. Tools to get BNF in that form:
- eliminate left recursion
- left factoring
- expanding non-terminals